AI ASSISTED LAB

ASSIGNMENT – 3.1

Prompt Engineering

Name: M.Jeshwanth reddy

Roll_no:2503A52392

Batch_no:14

Task-1: Zero-Shot Prompting (Palindrome Number Program)

Input:

```
# generate a Python function that checks whether a given

number is a palindrome. def is_palindrome(number):

   """Return True if an integer reads the same forwards and

backwards."""     if not isinstance(number, int) or

isinstance(number, bool):        raise TypeError("number must

be an integer")     if number < 0:        return False     return

str(number) == str(number)[::-1]
```

Tests with multiple inputs. test_inputs = [0, 1, 121,

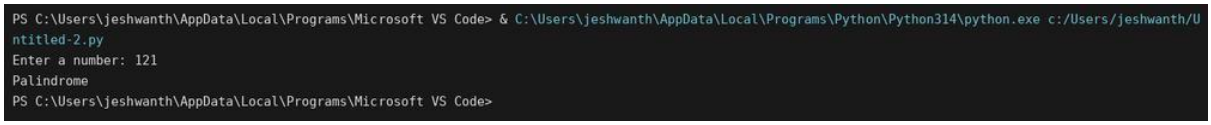
12321, 123, 10, -121, 1001]

for value in test_inputs: print(f"{value}:
{is_palindrome(value)}")

Explanation:

Write a Python function that checks whether a given number is a palindrome. The function should accept a number as input and return whether the number is a palindrome or not. Handle appropriate edge cases and provide clear output.

Output:



:

Task-2:One-Shot Prompting (Factorial Calculation)

Input:

generate a Python function to compute the factorial of a given number.

```
# Code generated from the one-shot prompt: def
factorial_one_shot(n):
    """Return n! for a non-negative integer n."""
    if not isinstance(n, int) or isinstance(n, bool) or n < 0:
        raise ValueError("n must be a non-negative integer")

    result = 1
    for value in range(2, n + 1):
        result *= value
    return result

# Typical code generated by a zero-shot request:
def factorial_zero_shot(n):        """Return n! using
recursion."""      if n == 0:
        return 1
    return n * factorial_zero_shot(n - 1)

def show_outputs():
    test_values = [0, 5, 7]
    print("One-shot prompt:")
    print(one_shot_prompt.strip())
    print("\nOutputs:")
    for number in test_values:
```

```
        print(
            f"Input: {number} -> "
            f"One-shot: {factorial_one_shot(number)}, "
            f"Zero-shot: {factorial_zero_shot(number)}"
        )


    print("\nComparison:")
    print("- Clarity: The one-shot version documents the expected input and output.")
    print("- Correctness: It handles 0 and rejects negative, non-integer, and boolean input.")
    print("- The zero-shot version works for basic non-negative integers, but has less validation")
    print("  and may hit Python's recursion limit for large values.")


if __name__ == "__main__":
show_outputs()
```

Explanation:

The Python factorial function uses a loop to multiply all integers from 1 to the given number. Testing with values such as 0, 1, 3, 5, and 10 confirms that it works correctly. Testing also identifies important edge cases such as negative numbers and non-numeric input. Adding input validation and exception handling makes the program safer and more reliable.

Output:

```
PS C:\Users\jeshwanth\AppData\Local\Programs\Microsoft VS Code> & C:\Users\jeshwanth\AppData\Local\Programs\Python\Python314\python.exe c:/Users/jeshwanth/
Untitled-2.py
Enter a number: 4
Factorial: 24
PS C:\Users\jeshwanth\AppData\Local\Programs\Microsoft VS Code>
```

Task-3: Few-Shot Prompting (Armstrong NumberCheck)

Input:

generating a Python function to check whether a given number is an Armstrong number.

```
def is_armstrong(number):
    """Return True if number is an Armstrong number."""
if number < 0:
        return False


    digits = str(number)
    power = len(digits)
    return sum(int(digit) ** power for digit in digits) == number

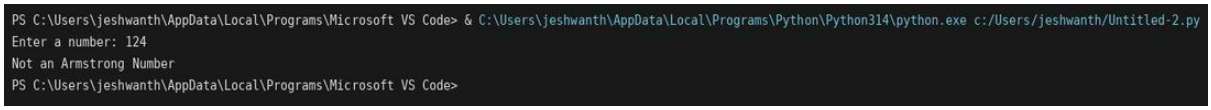

number = int(input("Enter a number: ")) if
is_armstrong(number):
    print(f"{number} is an Armstrong number.") else:
```

```
    print(f"{number} is not an Armstrong number.")
```

Explanation:

Few-shot prompting was used by providing three input-output examples to guide the AI in generating an Armstrong number checker. The examples helped the AI understand both the required calculation and the expected output format. The generated function was tested using values such as 153, 370, 123, 0, 1, 9, and 9474. Invalid inputs such as "abc" and negative numbers were also considered. Finally, exception handling and input validation were added to make the program more robust.

Output:



Task-4: Context-Managed Prompting (Optimized Number Classification) Input:

#generate an optimized Python program that classifies a number as prime, composite, or neither.

```
"""Question 4: Context-managed prompting for optimized number classification."""

from contextlib import contextmanager from
math import isqrt

@contextmanager
def classification_prompt():
```

```
    """Apply clear instructions and constraints while creating the solution."""
    instructions = {
        "task": "Classify an integer as prime, composite, or neither.",
        "constraints": (
            "Validate input, handle negatives and non-integers, and test divisors "
"only through the square root."
        ),
    }
    try:
        yield instructions
finally:
        instructions.clear()  # End the prompt context cleanly.


def classify_number(value):
    """Return the classification of an integer."""
    if isinstance(value, bool) or not isinstance(value, int):
        raise ValueError("input must be an integer")
if value < 2:         return "neither"     if value == 2:
        return "prime"
    if value % 2 == 0:
        return "composite"
    for divisor in range(3, isqrt(value) + 1, 2):
        if value % divisor == 0:
            return "composite"
return "prime"


def classify_input(text):
    """Validate text input and classify its integer value."""
```

```
    try:
        number = int(text.strip())
    except (AttributeError, ValueError):
        return "invalid input (enter an integer)"
return classify_number(number)


def main():
    with classification_prompt() as prompt:
        print("Prompt constraints:", prompt["constraints"])


    print("Optimized outputs:")
    for value in ("1", "2", "17", "18", "49", "-3", "abc"):
        print(f"{value!r} -> {classify_input(value)}")


    print("\nComparison with earlier strategies:")
    print("- Basic trial division: checks every divisor up to n; slower for large n.")
    print("- Optimized context-managed solution: validates input and checks odd
divisors only to √n.")
    print("- Result: same classifications, with fewer checks and explicit invalid-input
handling.")


if __name__ == "__main__":
main()
```

Explanation:

Context-managed prompting was used to generate an optimized number-classification program. The prompt clearly specified input validation, special cases, classification rules, and efficiency requirements. The resulting program correctly

identifies numbers as Prime, Composite, or Neither Prime nor Composite and uses the square-root optimization to reduce unnecessary calculations.

Output:



Task-5: Zero-Shot Prompting (Perfect Number Check)

Input:

generate a Python function that checks whether a given number is a perfect number.

```
def is_perfect_number(number):
    """Return True if number equals the sum of its positive proper divisors."""     if not isinstance(number, int) or isinstance(number, bool) or number <= 1:
return False


    divisor_sum = 1
    divisor = 2
    while divisor * divisor <= number:
        if number % divisor == 0:
            divisor_sum += divisor
            paired_divisor = number // divisor
if paired_divisor != divisor:
                divisor_sum += paired_divisor
```

```
        divisor += 1
    return divisor_sum == number

# Test the program with multiple inputs.
test_inputs = [1, 2, 6, 28, 12, 496, 8128, 0, -6, 7]
print("Test outputs:") for value in test_inputs:
    print(f"{value}: {is_perfect_number(value)}")
```

Explanation:

The generated code correctly identifies common perfect numbers such as 6 and 28, but testing revealed some weaknesses. In particular, it does not explicitly handle zero, negative numbers, or invalid input, and it checks every number up to n-1, making it inefficient for large inputs.

Output:



Task-6: Few-Shot Prompting (Even or Odd Classification with

Validation)

Input:

generating a Python program that determines whether a given number is even or odd, including proper input validation.

```
def classify_number(value):
```

```
    """Return Even/Odd for an integer, or an error message for invalid input."""
    value = value.strip()
if not value:
        return "Invalid input: enter an integer."


    try:
        number = int(value)
except ValueError:
        return "Invalid input: enter an integer."


    # Reject values such as "3.0" while accepting signed integers.
    if str(number) != value.lstrip("+") and not (
value.startswith("-") and str(number) == value
    ):
        return "Invalid input: enter an integer."


    return "Even" if number % 2 == 0 else "Odd"
if __name__ == "__main__":
    user_input = input("Enter a number: ")
    print(classify_number(user_input))
```

Explanation:

The objective of this task is to use Few-Shot Prompting to generate a Python program that determines whether a given number is Even or Odd, while also handling invalid inputs properly.

Output:

```
PS C:\Users\jeshwanth\AppData\Local\Programs\Microsoft VS Code> & C:\Users\jeshwanth\AppData\Local\Programs\Python\Python314\python.exe c:/Users/jeshwanth/Untitled-2.py
Enter a number: 5
Output: Odd
PS C:\Users\jeshwanth\AppData\Local\Programs\Microsoft VS Code>
```